

ARTEMIS: Adaptive Routing for Task-Efficient Multimodal Inference System

Vedaang Chopra
Georgia Institute of Technology
Atlanta, GA, USA

Chengyin Xu
Georgia Institute of Technology
Atlanta, GA, USA

Abstract—Vision-language models (VLMs) are vastly different in terms of accuracy and cost, making it economically inefficient to deploy a single model for all queries. Larger models are better at complex reasoning while smaller or more specialized models can handle routine captioning or OCR at a fraction of the cost. We present ARTEMIS, a cost-aware multimodal routing framework that learns to choose the most cost-efficient VLM adaptively while maximizing accuracy. ARTEMIS profiles multiple VLMs offline and assigns each input request with labels that combine correctness, semantic quality, and dollar cost. A lightweight router predicts the optimal model per request. We evaluate ARTEMIS on a new benchmark derived from the Cauldron dataset, and ARTEMIS achieves near-best-model accuracy at much lower cost, outperforming cascaded escalation and semantic routers. Our design builds a strong foundation for future systems that integrate runtime feedback loops and adaptive model scaling.

Index Terms—vision-language models, multimodal routing, cost-aware inference, systems, model selection

I. INTRODUCTION

Vision-language models (VLMs) have rapidly developed and now support a variety of tasks including visual question answering, diagram interpretation, optical character recognition (OCR), and more complex multimodal reasoning. However, current VLM deployments face a key challenge: no single model provides the best trade-off between accuracy and cost across all tasks.

Smaller VLMs are less expensive and sufficient for basic tasks such as captioning or simple OCR, while larger reasoning-focused models are more expensive but substantially more accurate for harder tasks. Enterprises with tight latency and budget constraints cannot afford to use large VLMs for every request. On the other hand, relying exclusively on cheaper models can cause significant accuracy degradation on complex workloads.

This creates a *model selection problem* at inference time: given a multimodal request, which model should be chosen to maximize accuracy per dollar?

Naive strategies such as always using the largest model, always using the cheapest model, or hand-crafted rule-based heuristics break down under real-world conditions where task types, difficulty, and budgets fluctuate. Prior work such as semantic routers, early-exit models, and cascaded escalation either:

- do not incorporate visual understanding,

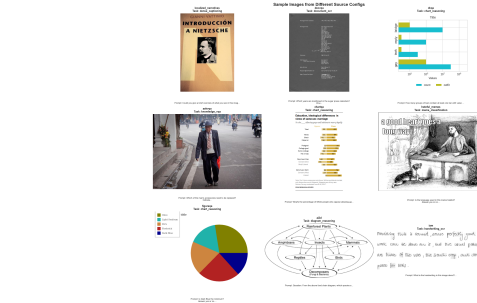


Fig. 1. Cauldron basic Dataset

- ignore monetary cost and focus solely on accuracy or latency, or
- are not adaptive at the per-sample level.

ARTEMIS addresses this gap by learning a *cost-aware, task-sensitive* router over a pool of heterogeneous VLMs.

A. Problem Statement

The problem formalization is as follows. Consider a set of deployed VLMs $\{M_1, \dots, M_K\}$ with:

- different inference costs (e.g., dollars per 1k tokens, GPU time), and
- different accuracy profiles across tasks (e.g., code and math, OCR, captioning).

Given an input multimodal request x with image and text, the system must choose a model index $k^*(x)$ such that:

- 1) the accuracy is maximized, and
- 2) the expected cost is minimized.

We aim to design a routing policy $f_\theta(x)$, defined by a lightweight router, which maps each input x to a model while optimizing the accuracy–cost trade-off.

In this paper, we:

- profile each candidate VLM on a multimodal benchmark derived from the Cauldron dataset,
- construct per-sample labels that combine correctness, dollar cost, and semantic quality, and
- train a router to predict the most cost-efficient model for each input.

Task Category	User Question	Large Model	Large Model Answer	Small Model	Small Model Answer
Captioning	"write the title for this image" 	Gemini-1.5-flash-001	Prada iPhone Case Black Leather iPhone Case with Prada Logo	Qwen-2.5-3B-Instruct	Prada Milano iPhone Case

Fig. 2. Case Study 1: Small VLMs achieve comparable accuracy at significantly lower cost for simple captioning tasks.

Task Category	User Question	Large Model	Large Model Answer	Small Model	Small Model Answer
Captioning	"What do you see?" 	Qwen-2.5-7B-Instruct Latency: 2.0s LLM-as-a-judge quality score: 88	The image is a black and white cartoon depicting an unusual scene inside a house. On the left, a person wearing a cap and carrying a bag appears to be entering the house. Inside, there are numerous animals, including goats, dogs, and what looks like a pig, scattered throughout the living room. The animals are either...	Qwen-2.5-3B-Instruct Latency: 1.0s LLM-as-a-judge quality score: 75	The cartoon depicts a scene in a living room where a woman is entering the room carrying a bag and a suitcase, indicating she might be moving in or out. The man sitting on the couch is surrounded by various animal figurines, including sheep, goats, and a dog, which are arranged on shelves and the floor. The animals appear to be interacting with each other or the man...

Fig. 3. Case Study 2: Performance comparison between Qwen-2.5-3B-Instruct and Qwen-2.5-7B-Instruct on a captioning task. The mid-sized model provides a more complete description at higher latency and cost, illustrating a trade-off where optimal model choice depends on input difficulty and budget constraints.

II. BACKGROUND

In this section we provide context for the ARTEMIS system, focusing on the opportunity for routing and our contributions relative to prior work.

A. Opportunity Analysis

Existing VLMs differ significantly in terms of accuracy and cost across task categories. For example, a large reasoning model may provide better performance on complex reasoning tasks, but be overkill for simpler queries. Conversely, a small model may be sufficient for easy captioning but fail on diagrams or multi-step reasoning.

By profiling a set of VLMs on VisionArena [4], we noticed patterns where:

- multiple models achieve similar accuracy but at very different cost, or
- a larger model is necessary to prevent severe accuracy drop.

To prove this point, we highlight 3 case studies below:

Case Study 1: For easier tasks such as routine captioning, smaller models are often sufficient and more concise than larger models despite being much cheaper (see Fig. 2). Always using larger VLM models would therefore lead to inefficiency.

Case Study 2: Some tasks see a trade-off where smaller models understand the visual scene but produce less complete answers than mid-sized models. The optimal choice depends on the user budget and per-sample difficulty (Fig. 3).

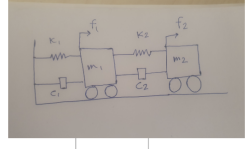
Task Category	User Question	General Model	General Model Answer Evaluation	Reasoning Model	Reasoning Model Answer Evaluation
Code	"how can i model this dynamic system in matlab? Plot ramp, impulse and step response" 	Qwen2.5-VL-7B-Instruct	LLM-as-a-judge Score: 75 The model answer provides a reasonable approach to solving the problem, including deriving equations and but the response is incomplete and doesn't directly show the responses to the step, ramp, and impulse inputs as requested.	Kimi-VL-A3B-Thinking-2506	LLM-as-a-judge Score: 35 The model answer spends a significant amount of time attempting to interpret the diagram and derive equations, but fails to solve the system as requested in the user question, making it largely irrelevant despite the effort to understand the system.

Fig. 4. Case Study 3: Comparison between a general-purpose VLM (Qwen2.5-VL-7B-Instruct) and a reasoning-focused model (Kimi-VL-A3B-Thinking-2506) on a coding task. The general model produces a mostly correct response, whereas the reasoning model hallucinates, showing that larger or reasoning models may not always work.

Case Study 3: For OCR-heavy or code and math reasoning tasks, specialized or larger models could outperform smaller ones. Surprisingly, reasoning models may not reliably improve outcomes due to hallucination. Therefore, correctness is input-dependent, and model size alone does not guarantee better performance (Fig. 4).

These case studies reveal the potential gains of routing: if we can adaptively route multi-modal inputs to the optimal model that maximizes accuracy while minimizing cost then we could achieve cost-efficiency.

B. Contributions

ARTEMIS makes the following contributions:

- **New multimodal routing benchmark.** We construct a new routing dataset derived from the Cauldron dataset and attach each sample with utility labels from different VLM models that combine model correctness and dollar cost. This results in a novel benchmark tailored for evaluating multimodal routing strategies under real-world settings.
- **Lightweight multimodal router design.** We design a lightweight router that fuses frozen vision and language encoders using a learned classifier to predict the most cost-effective VLM for each input. There is minimal inference overhead due to the lightweight architecture, allowing routing to scale over high workloads.
- **Systems integration.** We show how Artemis is positioned in a full online inference stack, demonstrating how its router, SLA-aware load balancer, and online monitoring combine to support multimodal workloads. We further discuss ideas such as autoscaling model instances, and continuous retraining based on online feedback as future work.

III. ARTEMIS DESIGN

In this section, we describe how ARTEMIS constructs the routing dataset, defines the optimal model label per example, and implements the router architecture within a multimodal inference system.

A. System Components

ARTEMIS is implemented as a modular system comprising five major components:

- **ARES (Automated Response Evaluation System).** ARES ingests Cauldron-derived multimodal datasets into PostgreSQL and stores prompts, images, model outputs, token usage, latency measurements, and evaluation metrics. It provides a clean relational schema (`vlm_samples`, `vlm_images`, `vlm_responses`, `vlm_evaluations`) that supports offline profiling and efficient router training.
- **Router.** The router predicts utility scores for each model and routing mode and exposes a simple API (`route()` and `route_batch()`). It returns both the predicted best model and the full reward vector, enabling downstream SLA-based decision making.
- **SLA-Aware Load Balancer.** The load balancer enforces system-level constraints such as global latency targets, throughput limits, and cost budgets. It may override router decisions if executing the chosen model would violate an SLA.
- **Inference Engine.** A unified model-execution layer that interacts with VLMs through an OpenAI-compatible API, tracking token-level billing and latency for every request. It supports multiple backends deployed using vLLM.
- **System API and Data Loop.** A FastAPI service exposes an OpenAI-compatible `/v1/chat/completions` endpoint. All routed queries and model responses are logged back into the database to support monitoring and future retraining.

This modularization cleanly separates offline profiling, utility modeling, SLA-aware scheduling, and model execution, enabling each component to evolve independently.

B. Accuracy and Routing Evaluation Set

We derive our routing benchmark from the Cauldron dataset [1], which is a collection of 40+ multimodal tasks such as OCR, scienceQA, VQA, captioning, and math. Each sample contains a question, image and ground truth.

To create the routing dataset, for each sample, we ran it across five different VLMs (DeepSeek-OCR, Qwen2.5-VL-3B, Qwen2.5-VL-7B, Qwen3-VL-8B-Thinking, Gemma3-27B), and record the following:

- Model output and its confidence
- Image and textual features: resolution, aspect ratio, prompt, question type
- Correctness: Exact match, F1 for MCQ
- Semantic quality: GLIDER (LLM-as-a-judge) score [2]
- Validity: empty outputs, refusals or errors
- Latency: inference time
- Cost metrics: USD cost based on input/output tokens [3]

As shown in Figure 5, Qwen2.5 VL-7B and Gemma3-27B records the highest accuracy on diagram reasoning (approximately 87%), while Qwen2.5 VL-3B performs better on rendered text OCR (90.2%) despite being much smaller,

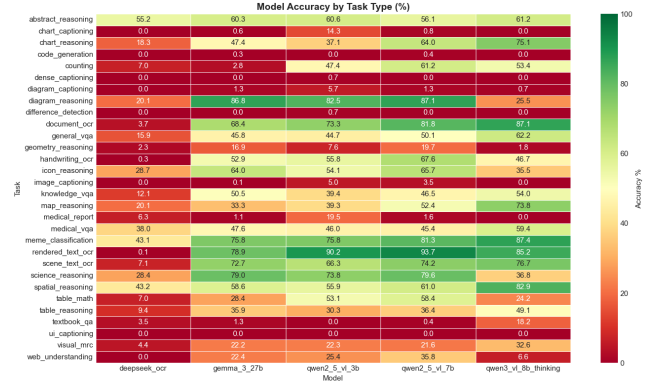


Fig. 5. Different VLMs achieve varying accuracies across task types, showing that no single model is the best. This highlights the need for adaptive routing.

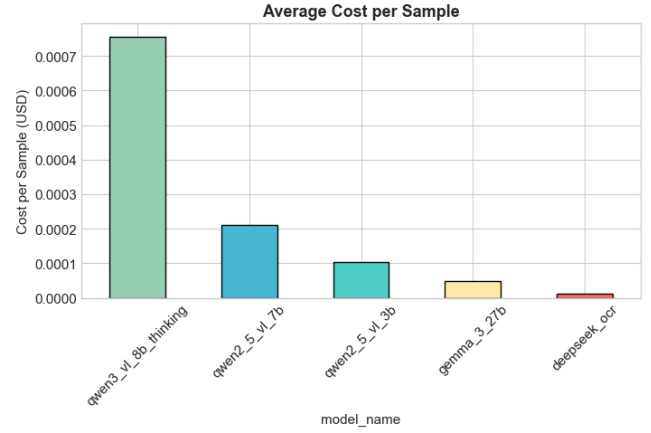


Fig. 6. Token-based pricing varies substantially across VLMs, which means that choosing the most cost-efficient model is important.

only slightly behind Qwen2.5 VL-7B (93.7%). Qwen3 VL-8B Thinking leads by a substantial amount on spatial reasoning (82.9%), but it is also the most expensive model. DeepSeek-OCR is not the top model, but it is also the cheapest (Figure 6) and fastest (Figure 7). These pattern show that accuracy and cost are not uniform across tasks and models, again highlighting the need for adaptive routing.

C. Optimal Model Selection

Artemis computes a utility reward score for every model-sample pair and considers the model that maximizes this score as optimal. This makes routing a supervised learning problem.

Each candidate VLM is profiled using four normalized metrics: (1) *Accuracy*, which is a combination of semantic quality determined by LLM-as-a-judge (GLIDER), correctness, and answer validity; (2) *Cost*, which is input and output per-token dollar cost from OpenRouter API billing rates; (3) *Latency*, measured as wall-clock inference time from vLLM; and (4) *Confidence*, which is from the model’s output token log-probabilities. All metrics are normalized, and hyperparameters are tuned to reflect the best performance.

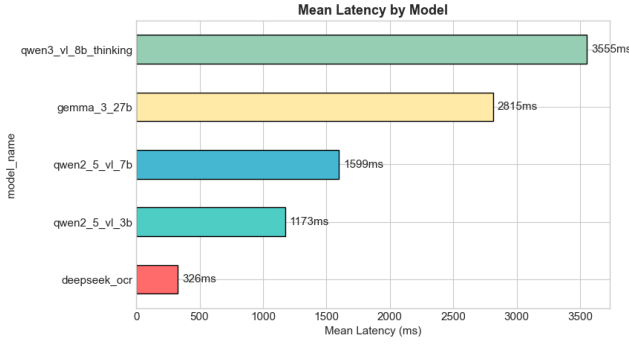


Fig. 7. Latency differs too amongst different VLMs

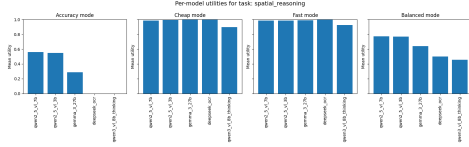


Fig. 8. Effect of Mode on model selection in dataset

Artemis supports four routing model modes—*accuracy*, *cheap*, *fast*, and *balanced*—each of them is defined by a different reward function $R_{i,k}$ over sample i and model k . The optimal model label is then

$$k_i = \arg \max_k R_{i,k}.$$

The reward functions for each mode are:

- **Accurate mode:** only prioritizes accuracy

$$R_{i,k} = \text{Accuracy}_{i,k}$$

- **Cheap mode:** selects accurate yet cost-efficient models

$$R_{i,k} = \text{Accuracy}_{i,k} - 0.7 \cdot \text{Cost}_{i,k}$$

- **Fast mode:** selects faster models while accounting for accuracy

$$R_{i,k} = \text{Accuracy}_{i,k} - 0.7 \cdot \text{Latency}_{i,k}$$

- **Balanced mode:** multi-objectives and balances between accuracy, confidence, cost, and latency

$$R_{i,k} = \text{Accuracy}_{i,k} + 0.3 \cdot \text{Confidence}_{i,k} - 0.3 \cdot \text{Cost}_{i,k} - 0.3 \cdot \text{Latency}_{i,k}$$

For each mode, Artemis generates both *hard labels* (the maximizing model k_i) and *soft labels* obtained via a softmax over $R_{i,k}$:

$$p_{i,k} = \frac{e^{R_{i,k}}}{\sum_j e^{R_{i,j}}}.$$

For the loss function, the router is trained using cross-entropy on the hard labels and KL-divergence on the soft utility distributions. This dual-objective learning encourages both decisive decision making and better calibration and robustness.

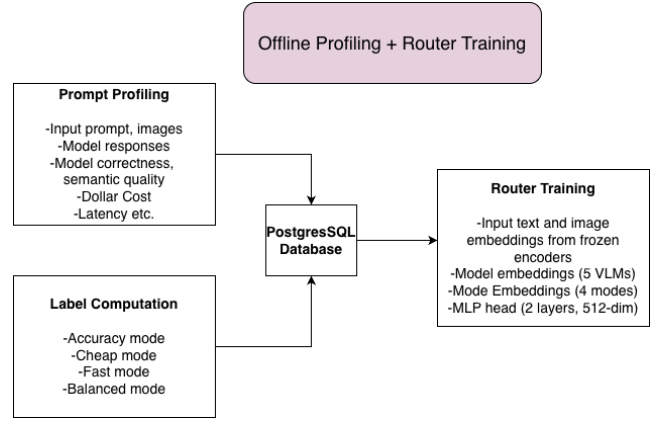


Fig. 9. Offline Profiling: Candidate VLM responses and labels are stored, then used as training data for router

D. System Architecture

ARTEMIS consists of components for both offline profiling and online inference.

Offline Profiling: Artemis’ offline profiling pipeline records the raw data from multimodal prompts, model responses, compute the optimal model labels based on accuracy-cost tradeoff, and undergoes supervised training for the VLM router, which will subsequently be deployed online. (Figure 9)

1) *Prompt Profiling:* We use PostgreSQL database to record metadata for every image–prompt pair across all candidate VLMs. For each query, the system stores model responses, correctness and semantic quality metrics, dollar cost and latency, and multimodal metadata such as image resolution and prompt length.

2) *Utility-Based Label Computation:* Artemis supports four routing modes as outlined under *Optimal Model Section* section—*accuracy*, *cheap*, *fast*, and *balanced*. For every sample and every mode, the system produces a label. This prepares the dataset for the subsequent offline supervised router training.

3) *Offline Router Training:* The profiling dataset trains a compact multimodal router that predicts which VLM to choose from for a given query. The router uses frozen CLIP and DistilBERT encoders for image and text features, fuses embeddings via a lightweight MLP architecture, and outputs a probability distribution over candidate VLMs. For each mode (accuracy, cheap, fast, balanced), we will train a router. Training optimizes on cross-entropy on the hard labels and KL-divergence on the soft distributions, allowing for both correctness and flexibility. Only the fusion and classifier layers are learned, therefore training and inference will be relatively inexpensive.

Online Inference: Once trained offline, Artemis acts as a real-time inference platform that routes multimodal requests to the most optimal VLM given cost, latency, and accuracy SLAs. This online inference stage consists of a lightweight router inference engine, an SLA-aware load balancer, VLM execution queues, and online monitoring. (Figure 10)

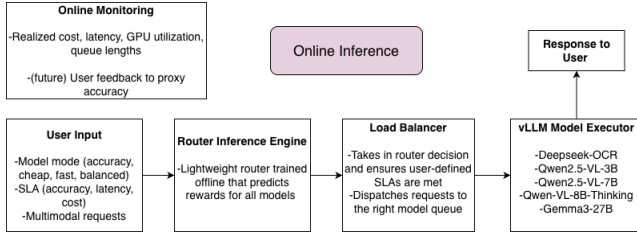


Fig. 10. Online Inference: Router is deployed online with load balancer to ensure SLAs are met. Online monitoring is in place.

4) *Router Inference Engine*: The router receives a batch of multimodal request and a user-selected routing mode, which could be *accuracy*, *cheap*, *fast*, or *balanced*. It will run the multimodal request through the chosen mode, and outputs a probability distribution over the candidate VLMs. For each candidate VLMs, it will also record down the estimated latency, dollar cost, and confidence score for that request. It will then determine the top model choice based on the highest probability of being chosen. All these information will then be passed on to the load balancer for the final routing decision given real-time constraints.

5) *SLA-Aware Load Balancer*: The load balancer takes in the router’s proposed allocation and evaluates whether the batch satisfies user-defined SLAs for total latency, dollar cost, and expected accuracy. The user may prioritize and rank these objectives (e.g., $\text{latency} > \text{cost} > \text{accuracy}$), and the balancer adjusts routing decisions accordingly. If an SLA is violated, the balancer will go through the requests in order of the queue and replaces the VLM candidate models that are cheaper, faster, or more accurate, depending on which constraint is not met. If satisfying one SLA breaks another, the system will alert the user before proceeding. It is hoped that the router already implicitly learns how to satisfy accuracy, cost and latency requests based on the different modes. The load balancer therefore serves as a final guardrail to make sure that the user would be satisfied with the online inference service.

6) *vLLM Backend for Candidate Models*: After routing decisions are finalized, requests are dispatched into queues corresponding to the five VLMs, which are deployed using vLLM. Each model runs in parallel on different GPUs.

7) *Online Monitoring*: Artemis also logs routing decisions, predicted and actual cost, latency, GPU utilization, and queue lengths. While ground-truth accuracy may not be available in real-time online, the system could in future ask for lightweight user feedback to proxy user satisfaction. Future work could include triggering router retraining when it detects drift in data distribution or performance degradation.

IV. EVALUATION

V. IMPLEMENTATION

ARTEMIS is implemented as a Python 3.10 system with modular packages for the router, load balancer, inference engine, and ARES evaluation pipeline.

A. Software Stack

We use PyTorch for the router model, SQLAlchemy for database access, and FastAPI for serving. Offline profiling uses vLLM for executing VLMs in parallel across GPU nodes. The entire stack is containerized using Docker, and a provided Compose configuration launches the ARTEMIS API together with a PostgreSQL instance for experimentation.

B. Database and Evaluation Pipeline

ARES defines the normalized relational schema used throughout ARTEMIS. The profiling stage populates the following tables:

- `vlm_samples`: input prompts and metadata
- `vlm_images`: associated image blobs
- `vlm_responses`: model outputs, token usage, latency, and cost
- `vlm_evaluations`: correctness, GLIDER scores, and validity flags

A denormalized SQLite cache is automatically generated for faster training throughput.

C. Router and Load Balancer Services

The router is loaded as a long-lived model within the FastAPI server and supports both single-request and batch routing. The load balancer maintains per-model state, including queue depth, recent latency samples, and estimated throughput. It can operate in router-only mode or override the router when SLA violations are detected.

D. Serving Path

At inference time, each request flows through (1) router prediction, (2) SLA-aware adjustment, (3) dispatch to the selected VLM backend, and (4) response logging into ARES. This unified serving path simplifies debugging, monitoring, and future retraining.

We evaluate ARTEMIS on the multimodal routing benchmark that we constructed from the Cauldron dataset. To the best of our knowledge, no existing benchmark provides optimal model label based on different accuracy-cost tradeoffs on heterogeneous VLMs, therefore our dataset serves as the evaluation benchmark.

E. Baselines

We compare ARTEMIS against four distinct routing strategies:

- **Always-largest model**: routes every request to the most expensive model. This assumes that the most expensive model is the most accurate.
- **Always-cheapest model**: sends all inputs to the smallest VLM, minimizing cost but at the expense of accuracy.
- **CascadeFlow**: a confidence-based cascade that escalates to larger models only when necessary. It saves cost with cascades, but it is not learned or trained, as it does not do per-sample profiling. We configure CascadeFlow with the same models and compare it directly to our trained router.

TABLE I
CASCADEFLOW PERFORMANCE SUMMARY ON 100 AND 1000 QUERIES.

Samples	Accuracy	Total Cost (\$)	Avg. Latency (ms)
100	73.00%	0.0004	1572.9
1000	32.30%	0.0101	658.1

CascadeFlow achieves a mean performance of **0.43** and a mean cost of **3.0×10^{-5} USD/query**.

- **Semantic Router (Aurelio):** a text-only embedding classifier that classifies queries into models based on semantic similarity. It does not have visual understanding skills, and achieves a mean performance of **0.668** and a mean cost of **8.3×10^{-5} USD/query**.

These baselines show the limitations of relatively static routing or routing that does not account for cost.

F. Metrics

We report the following metrics across all routing policies:

- **Performance:** a combination of correctness, semantic quality using GLIDER scores [2], and output validity.
- **Dollar cost per query:** computed using token-based pricing obtained from the OpenRouter API [3].
- **Accuracy–cost trade-off:** we aim to achieve pareto frontier

G. Results

We first evaluate the end-to-end performance of the CascadeFlow baseline on two query loads (100 and 1000 samples). Since ARTEMIS was not enabled in this experiment, the results reported here establish the reference point that the router must eventually surpass. Table I summarizes the accuracy, dollar cost, and latency characteristics of CascadeFlow under both evaluation scales.

The 100-sample evaluation shows that CascadeFlow achieves relatively high accuracy (73%) for low throughput settings. However, when scaled to 1000 queries, its accuracy drops to 32.3%, indicating that the heuristic escalation policy becomes significantly less effective as task diversity increases. Cost remains low in both cases, reflecting that CascadeFlow escalates to more expensive models only when its confidence threshold is not met.

1) *Router Task Classification Performance:* Although ARTEMIS was not used for routing in this experiment, we evaluate its task-classification head separately to understand its underlying modeling capability. Over 1000 labeled queries, the router achieves a macro F1-score of 0.97 and a weighted F1-score of 0.98 across 30 task categories. The full breakdown is shown in Fig. ?? (not included here for brevity).

To understand how the router distributes inference load, Fig. ?? reports model-selection frequencies across the 996 routed samples. The router shows balanced utilization across the five candidate VLMs:

- **deepseek_ocr**: 235 selections
- **qwen2.5-vl-3b**: 212 selections
- **qwen2.5-vl-7b**: 192 selections

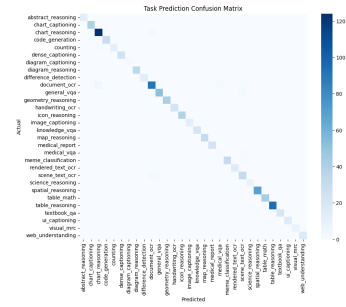


Fig. 11. Confusion Matrix on Test Set of Benchmark

- **qwen3-vl-8b-thinking**: 189 selections
- **gemma-3-27b**: 168 selections

This distribution indicates that the router is capable of differentiating across task types and selecting models with varying cost-capability profiles, even though end-to-end routing was disabled in the present CascadeFlow evaluation.

ARTEMIS is expected to lie on the upper-left frontier of the accuracy–cost space, achieving near-best-model performance at significantly reduced cost relative to static baselines. Different routing modes (*accurate*, *cheap*, *fast*, *balanced*) allow shifting along this trade-off surface, enabling application-specific optimization.

Quantitative Summary (to be completed):

- Mean performance: 0.14257
- Mean cost per query: \$0.000343
- Mean latency: 408.877 s

H. Discussion

The CascadeFlow results reveal two important observations. First, while CascadeFlow performs well on small batches, its accuracy degrades sharply with increasing workload diversity. This suggests that static confidence-based escalation is insufficient when queries span a wide range of difficulty levels. Second, although CascadeFlow maintains extremely low monetary cost, its accuracy–cost trade-off is suboptimal for larger query volumes.

The router analysis shows strong task-recognition capability (F1 0.97–0.98), confirming that ARTEMIS is capable of learning fine-grained distinctions across multimodal query categories. The model-selection histogram further indicates that the router learns a meaningful allocation strategy—leaning on OCR-specialized models for text-heavy samples and selecting larger models for reasoning or diagram-based tasks.

However, because routing was disabled in the CascadeFlow run, the ARTEMIS end-to-end cost and accuracy values do not reflect actual system behavior and should not be interpreted as performance metrics. Instead, these results serve to validate the internal router model and motivate the next stage of evaluation, where ARTEMIS is fully integrated with the execution backend and evaluated against CascadeFlow on identical query batches.

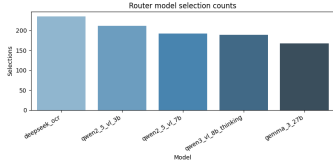


Fig. 12. Selection from the ARTEMIS router post 1000 runs

Overall, these preliminary results highlight the opportunity for ARTEMIS to surpass CascadeFlow by leveraging its strong task-awareness and adaptive model selection once routing is enabled in the full system.

VI. RELATED WORK

Existing efforts to improve the efficiency of LLMs mostly fall into two categories: intra-model adaptivity and inter-model routing.

Intra-model adaptivity. Vision Mixture-of-Experts architectures such as MoVA [5] demonstrate sparse gating mechanisms that activate only a subset of experts depending on the input. Elastic model designs such as Flextron [6] select subnetworks based on difficulty and cost targets. Early-exit techniques like E³ [7] allow inference to terminate early on easier samples. While these methods provide input-adaptive compute within one model, they do not address the challenge of selecting among different VLMs with different accuracy and cost profiles.

Inter-model adaptivity. These works focus on selecting the most optimal model. Semantic routers such as vLLM Semantic Router [8] and RouteLLM [9] predict query intent mostly using text-only LLMs, and cascaded approaches such as CascadeFlow [10] escalate to larger models only when needed. However, these systems are either mostly on text, ignore real dollar cost, or rely on static rules associated with coarse task categories.

Systems perspective. There has been work on autoscaling, and heterogeneous deployment of different models, but current approaches do not integrate accuracy–cost trade-offs into routing. As VLM workloads become more diverse and budgets more limited, there has to be a system in place to decide which model to run.

Positioning Artemis. Artemis differs from these prior efforts by framing multimodal model selection as a *utility-supervised routing problem* over a set of VLMs with varied strengths. It integrates model accuracy and dollar cost into the decision process adaptively for each input.

VII. CONCLUSION

This work presented ARTEMIS, a cost-aware routing system for multimodal inference over heterogeneous vision–language models. By profiling multiple VLMs on a Cauldron-derived benchmark and learning a utility-based routing policy, ARTEMIS aims to select the most suitable model for each query under accuracy, latency, and cost constraints. Our design combines offline profiling, a lightweight text-based

router, and an SLA-aware load balancer to support adaptive per-sample decision making. Preliminary experiments demonstrate that ARTEMIS learns fine-grained task distinctions and meaningful model preferences, positioning it to improve upon static or heuristic routing baselines once fully integrated into the inference pipeline.

VIII. FUTURE WORK

Several directions remain to make ARTEMIS a complete, production-ready multimodal serving system.

Online learning and feedback integration. The current router is trained entirely offline. Incorporating online feedback—such as realized latency, actual token expenditure, or lightweight user acceptance signals—would allow ARTEMIS to adapt to workload drift, data distribution shifts, and model degradation. Trigger-based retraining or continuous refinement offers a promising path toward long-term robustness.

SLA-driven autoscaling and capacity management. The load balancer presently operates over a fixed set of model replicas. In practical deployments, the system should dynamically scale replicas based on traffic patterns, queue lengths, and predicted SLA violations. Coupling routing decisions with autoscaling policies could significantly improve overall throughput and cost efficiency.

Extending the model pool and modality coverage. ARTEMIS currently routes across five VLMs, but multimodal applications increasingly rely on specialized models (e.g., OCR-focused, math/code solvers, speech-capable models). Expanding the candidate pool and supporting heterogeneous modalities would enable ARTEMIS to serve a broader range of workloads and evaluate richer trade-offs between accuracy and cost.

Taken together, these directions highlight ARTEMIS as a step toward a fully adaptive, budget-aware multimodal inference platform capable of optimizing system performance under real-world constraints.

REFERENCES

- [1] H. Laurençon, L. Tronchon, M. Cord, and V. Sanh, “What matters when building vision-language models?,” *arXiv preprint arXiv:2405.02246*, 2024.
- [2] D. Deshpande, S. S. Ravi, S. CH-Wang, B. Mielczarek, A. Kannappan, and R. Qian, “GLIDER: Grading LLM Interactions and Decisions using Explainable Ranking,” *arXiv preprint arXiv:2412.14140*, 2024.
- [3] OpenRouter, “OpenRouter: unified API for 400+ AI models and transparent per-token pricing,” accessed December 7, 2025. <https://openrouter.ai/pricing>.
- [4] C. Chou, L. Dunlap, K. Mashita, K. Mandal, T. Darrell, I. Stoica, J. E. Gonzalez and W.-L. Chiang, “VisionArena: 230K Real World User–VLM Conversations with Preference Labels,” *arXiv preprint arXiv:2412.08687*, 2024.
- [5] Z. Zong, B. Ma, D. Shen, G. Song, H. Shao, D. Jiang, H. Li, and Y. Liu, “MoVA: Adapting Mixture of Vision Experts to Multimodal Context,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. *arXiv preprint arXiv:2404.13046*.
- [6] R. Cai, M. J. Shafiei, M. M. Kothapalli, G. Khanna, P. Molchanov, N. F. Duffield, and S. M. Fritz, “FLEXTRON: Many-in-One Flexible Large Language Model,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. *arXiv preprint arXiv:2406.10260*.

- [7] A. P. Iyer, E. Zhang, C. Zhang, and K. Olukotun, “Improving DNN Inference Throughput Using Practical, Per-Input Compute Adaptation,” in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP)*, 2024.
- [8] vLLM Project, “vLLM Semantic Router: Intelligent Mixture-of-Models Routing for LLM Inference,” documentation, accessed December 7, 2025. Available at: <https://vllm-semantic-router.com/docs/intro>.
- [9] I. Ong, L. Zheng, H. Li, K. Zhang, L. Zheng, L. Zheng, S. Xu, H. Liu, P. Liang, and H. Zhang, “RouteLLM: Learning to Route LLMs with Preference Data,” arXiv preprint arXiv:2406.18665, 2024.
- [10] Lemony AI, “CascadeFlow: A Cost-Optimization Model-Cascading Framework for LLM Inference,” project documentation and source code, accessed December 7, 2025. Available at: <https://github.com/lemony-ai/cascadeflow>.